1) Calcular si un nombre és primer
- Enunciat: Escriu una funció que diga si n és primer

- Solució (JS):

```
function esPrimer(n) {
  if (n < 2) return false;
  if (n === 2) return true;
  if (n % 2 === 0) return false;
  for (let i = 3; i * i <= n; i += 2) {
    if (n % i === 0) return false;
  }
  return true;
}

// Exemples
console.log(esPrimer(17)); // true
console.log(esPrimer(15)); // false
```

2) Factorial (iteratiu)
- Enunciat: Calcula el factorial de n utilitzant un mètode iteratiu (no recursiu).

- Solució (JS):

```
function factorial(n) {
  if (n < 0) throw new Error("n ha de ser no negatiu");
  let res = 1;
  for (let i = 2; i <= n; i++) res *= i;
  return res;
}

// Exemples
console.log(factorial(5)); // 120
```

3) Màxim entre tres nombres
- Enunciat: Donats tres enters a, b, c, retorna el màxim.

- Solució (JS):

```
function maxTres(a, b, c) {
  let m = a;
  if (b > m) m = b;
  if (c > m) m = c;
  return m;
}

// Exemples
console.log(maxTres(3, 9, 5)); // 9
```

IES MARIA ENRIQUEZ

4) Divisors d'un nombre
- Enunciat: Imprimeix tots els divisors de n en ordre ascendent

- Solució (JS):

```
function divisors(n) {
  n = Math.abs(n);
  let resultat = [];
  for (let d = 1; d * d <= n; d++) {
    if (n % d === 0) {
      resultat.push(d);
      if (d !== n / d) resultat.push(n / d);
    }
  }
  resultat.sort((x, y) => x - y);
  return resultat;
}

// Exemple
console.log(divisors(28)); // [1, 2, 4, 7, 14, 28]
```

5) Comptar vocals en una cadena
- Enunciat: Donada una cadena s, compta les vocals (a, e, i, o, u)

- Solució (JS):

```
function comptaVocales(s) {
  const v = new Set(['a','e','i','o','u','A','E','I','O','U']);
  let compta = 0;
  for (let i = 0; i < s.length; i++) {
    if (v.has(s[i])) compta++;
  }
  return compta;
}

// Exemples
console.log(comptaVocales("Hola món")); // 3 (o, a, o)
```

6) Comptar zeros al final de n! (núm de zeros de factorització)
- Enunciat: Donat n, determina quants zeros té n! a la dreta (màxim nombre de zeros).

- Solució (JS):

```
function zerosFactorial(n) {
  let zeros = 0;
  let p = 5;
  while (p <= n) {
```

IES MARIA ENRIQUEZ

```
    zeros += Math.floor(n / p);
    p *= 5;
  }
  return zeros;
}

// Exemples
console.log(zerosFactorial(10)); // 2
```

7) Comprovació de coprimes amb l'algorisme d'Euclides
- Enunciat: Donats a i b, escriu si són coprims (GCD(a, b) == 1) utilitzant l'algorisme d'Euclides. (divisions successives)

- Solució (JS):

```
function areCoprime(a, b) {
  a = Math.abs(a);
  b = Math.abs(b);
  while (b !== 0) {
    const t = a % b;
    a = b;
    b = t;
  }
  return a === 1;
}

// Exemples
console.log(areCoprime(35, 16)); // true
console.log(areCoprime(21, 14)); // false
```

8) Comprovar si un nombre és perfet (divisors exclosos igual a n)
- Enunciat: Donat n, verifica si és un nombre perfecte (la suma dels seus divisors exclosant n és igual a n).

- Solució (JS):

```
function esPerfet(n) {
  if (n <= 1) return false;
  let suma = 1;
  for (let d = 2; d * d <= n; d++) {
    if (n % d === 0) {
      suma += d;
      if (d !== n / d) suma += n / d;
    }
  }
  return suma === n;
}
```

IES MARIA ENRIQUEZ

```
// Exemples
console.log(esPerfet(6)); // true (1+2+3)
console.log(esPerfet(28)); // true (1+2+4+7+14)
```

9) Suma recursiva de 1 a n
- Enunciat: Calcula la suma dels naturals de 1 a n de forma recursiva.

- Solució (JS):

```
function sumaRec(n) {
  if (n <= 0) return 0;
  return n + sumaRec(n - 1);
}

// Exemples
console.log(sumaRec(5)); // 15
```

10) Invertir una cadena recursivament
- Enunciat: Donada una cadena s, retorna la mateixa cadena però al revés, implementat de forma recursiva.

- Solució (JS):

```
function invertirRec(s) {
  if (s.length <= 1) return s;
  return s[s.length - 1] + invertirRec(s.slice(0, -1));
}

// Exemples
console.log(invertirRec("abcdef")); // "fedcba"
```

11) Comptar particions palíndromes d'una cadena
Enunciat:
Donada una cadena s, calcula el nombre de formes diferents de dividir s en substrings que siguin palindròmes. Cada partició és una manera de dividir trossos contigus en substrings palindròmics. Ex: "aab" té 2 particions: ["a","a","b"] i ["aa","b"].

```
Solució (JavaScript):
function countPalindromicPartitions(s) {
  const n = s.length;
  // pal[i][j] és true si s[i..j] és palíndrom
  const pal = Array.from({ length: n }, () => Array(n).fill(false));
  // Precompute palindromes
  for (let i = n - 1; i >= 0; i--) {
    for (let j = i; j < n; j++) {
```

IES MARIA ENRIQUEZ

```
      if (s[i] === s[j] && (j - i <= 2 || pal[i + 1][j - 1])) {
        pal[i][j] = true;
      }
    }
  }
  // dp[i] és el nombre de particions de s[i..n-1]
  const dp = new Array(n + 1).fill(0);
  dp[n] = 1;
  for (let i = n - 1; i >= 0; i--) {
    let sum = 0;
    for (let j = i; j < n; j++) {
      if (pal[i][j]) {
        sum += dp[j + 1];
      }
    }
    dp[i] = sum;
  }
  return dp[0];
}

// Exemple
console.log(countPalindromicPartitions("aab")); // 2
console.log(countPalindromicPartitions("aaa"));     //     4:     ["a","a","a"],
["a","aa"], ["aa","a"], ["aaa"]
```

12) Comptar camins en una graella amb obstacles
Enunciat:
Donada una m×n graella amb obstacles (cells bloquejades), calcula el nombre de camins des de la cantonada superior esquerra (0,0) fins a la cantonada inferior dreta (m-1,n-1), movent-te només cap a la dreta o cap avall. Representa obstacles com un conjunt de cadenes "i,j" o una matriu booleana obstacles[i][j] = true.

```
Solució (JavaScript):
function countPathsWithObstacles(obstacles) {
  const m = obstacles.length;
  if (m === 0) return 0;
  const n = obstacles[0].length;
  if (obstacles[0][0]) return 0;

  const dp = Array.from({ length: m }, () => Array(n).fill(0));
  dp[0][0] = 1;

  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      if (obstacles[i][j]) {
        dp[i][j] = 0;
```

IES MARIA ENRIQUEZ

```
        continue;
      }
      if (i > 0) dp[i][j] += dp[i - 1][j];
      if (j > 0) dp[i][j] += dp[i][j - 1];
    }
  }
  return dp[m - 1][n - 1];
}

// Exemple
const grid1 = [
  [false, false, false],
  [false, true,  false],
  [false, false, false]
];
console.log(countPathsWithObstacles(grid1)); // 2
```

13) Longitud de la millor subcadena sense caràcters repetits
Enunciat:
Donada una cadena s, troba la longitud de la subcadena més llarga sense caràcters repetits.

```
Solució (JavaScript):
function lengthOfLongestUniqueSubstring(s) {
  const pos = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    if (pos.has(s[right]) && pos.get(s[right]) >= left) {
      left = pos.get(s[right]) + 1;
    }
    pos.set(s[right], right);
    best = Math.max(best, right - left + 1);
  }
  return best;
}

// Exemple
console.log(lengthOfLongestUniqueSubstring("abcabcbb")); // 3 ("abc")
console.log(lengthOfLongestUniqueSubstring("bbbbb"));    // 1
```

14) Generació de totes les seqüències vàlides de parèntesis
Enunciat:
Donat un nombre n de parells, genera totes les seqüències vàlides de parèntesis amb n parells '(' i ')' de manera que sempre estiguin balancejades.

```
Solució (JavaScript):
function generateParenthesis(n) {
```

IES MARIA ENRIQUEZ

```
  const res = [];
  function backtrack(curr, open, close) {
    if (curr.length === 2 * n) {
      res.push(curr);
      return;
    }
    if (open < n) backtrack(curr + '(', open + 1, close);
    if (close < open) backtrack(curr + ')', open, close + 1);
  }
  backtrack('', 0, 0);
  return res;
}

// Exemple
console.log(generateParenthesis(3));
// [ "((()))","(()())","(())()","()(())","()()()" ]
```

15) Màxim sum subseqüència amb espai no adyacents
Enunciat:
Dada una successió d'entitats amb valors enter, troba el màxim sumatori d'una subseqüència no adjacent (no es poden escollir elements contigus). No es tracta de sol·licitar ordre de números en la seqüència, només la suma òptima.

Solució (JavaScript):

```
function maxNonAdjacentSum(nums) {
  if (nums.length === 0) return 0;
  let incl = Math.max(0, nums[0]); // millor suma que inclou el primer
  let excl = 0; // millor suma que exclou
  for (let i = 1; i < nums.length; i++) {
    const newExcl = Math.max(excl, incl);
    incl = excl + nums[i];
    excl = newExcl;
  }
  return Math.max(incl, excl);
}

// Exemple
console.log(maxNonAdjacentSum([3, 2, 7, 10])); // 13 (3 + 10)
console.log(maxNonAdjacentSum([3, 2, 5, 10, 7])); // 15 (3 + 5 + 7)
```

16) Enunciat:
Implementar una cerca binària recursiva en un vector ordenat de manera ascendent. La funció ha de retornar l'índex de l'element buscant si existeix, o -1 si no és present. La cerca es divideix en dos subproblems reduint a la meitat el rang de recerca en cada pas.

Descripció de la solució:

IES MARIA ENRIQUEZ

- Si el rang és invàlid (left > right), l'element no està present i retornem -1.
- Es calcula l'índex del mig: mid = floor((left + right) / 2).
- Si l'element buscant és igual a arr[mid], retornem mid.
- Si és menor que arr[mid], busquem a la meitat esquerra: left … mid-1.
- Si és més gran, busquem a la meitat dreta: mid+1 … right.
- La recursivitat continua fins a trobar l'element o fins que el range sigui invàlid.

Explicació detallada:
- Ventatge principal: cada pas descarta la meitat de l'espai possible, per tant la complexitat és O(log n).
- Cas base: rang invàlid (left > right) o element trobat en arr[mid].
- Cas recursiu: segons la comparació amb arr[mid], cridem recursivament la cerca en la meitat pertinent.
- Es pot exposar amb paràmetres opcionals left i right per inicialitzar com left = 0 i right = arr.length - 1.

Codi JavaScript (recursiu):

```
function binarySearch(arr, target, left = 0, right = arr.length - 1) {
  if (left > right) {
    return -1; // no trobat
  }

  const mid = Math.floor((left + right) / 2);

  if (arr[mid] === target) {
    return mid;
  } else if (target < arr[mid]) {
    return binarySearch(arr, target, left, mid - 1);
  } else {
    return binarySearch(arr, target, mid + 1, right);
  }
}

// Exemple d'ús
const v = [1, 3, 4, 6, 8, 10, 12];
console.log(binarySearch(v, 6));  // 3 (l'element 6 està a l'índex 3)
console.log(binarySearch(v, 7));  // -1 (no present)
console.log(binarySearch(v, 1));  // 0
console.log(binarySearch(v, 12)); // 6
```

Notes:
- L'array ha de ser ordenat per garantir que la cerca sigui correcta.
- Aquesta versió és recursiva; una versió iterativa també és possible però no és l'objectiu aquí.
- Si vols gestionar arrays de tipus diferents (com enters i floats), funciona igual sempre que l'ordre sigui ascendent i les comparacions siguin coerents.

IES MARIA ENRIQUEZ